

TP547 Trabalho 4

José Antonio e Alfredo

José Antonio García Ocaña Alfredo Jesús Arbolaez Fundora

Maio de 2026

Contents

1	Exercício 1 — Cadeia de Markov: Sensor IoT	2
1.1	Introdução	2
1.2	a) Diagrama de Transição de Estados	2
1.3	b) Matriz de Transição	2
1.4	c) Probabilidade de Absorção no 4º Ciclo	2
1.5	d) Matriz Fundamental	3
1.6	e) Número Médio de Ciclos até a Absorção	3
1.7	f) Probabilidade de Absorção	3
2	Exercício 2 — Simulação de Consultório Médico	4
2.1	Descrição do Sistema	4
2.2	a) Identificação dos Componentes do Sistema	4
2.3	b) Diagrama ACD	5
2.4	c) Fases de Simulação (Abordagem de Três Fases)	5
2.5	d) Dinâmica da Simulação	5
2.6	e) Tabela de Simulação Manual	6
2.7	f) Cálculo das Métricas	7
2.8	Discussão dos Resultados	7
A	Implementação em Python — Exercício 1	8

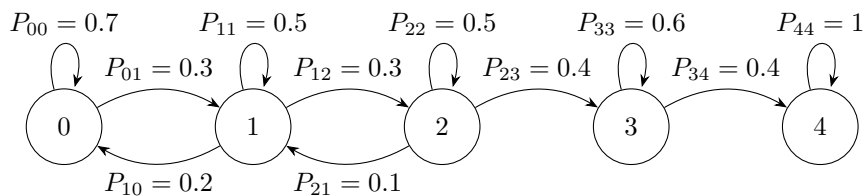
1 Exercício 1 — Cadeia de Markov: Sensor IoT

1.1 Introdução

Este trabalho analisa uma cadeia de Markov que modela os estados de operação de um sensor em uma rede IoT. O sistema possui cinco estados possíveis, onde o estado 4 é um estado absorvente.

- Estado 0: Sensor totalmente operacional
- Estado 1: Sensor com pequena degradação
- Estado 2: Sensor com falha moderada
- Estado 3: Sensor em modo crítico
- Estado 4: Sensor queimado (falha total)

1.2 a) Diagrama de Transição de Estados



1.3 b) Matriz de Transição

$$P = \begin{bmatrix} 0.7 & 0.3 & 0 & 0 & 0 \\ 0.2 & 0.5 & 0.3 & 0 & 0 \\ 0 & 0.1 & 0.5 & 0.4 & 0 \\ 0 & 0 & 0 & 0.6 & 0.4 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

1.4 c) Probabilidade de Absorção no 4º Ciclo

A probabilidade de atingir o estado 4 após exatamente 4 ciclos partindo do estado 0 é dada pelo elemento $(P^4)_{0,4}$:

$$p^{(4)} = p^{(0)} \cdot P^4, \quad p^{(0)} = [1 \ 0 \ 0 \ 0 \ 0]$$

$$P^4 = \begin{bmatrix} 0.3907 & 0.3294 & 0.1827 & 0.0828 & 0.0144 \\ 0.2344 & 0.2817 & 0.2394 & 0.1602 & 0.0843 \\ 0.0532 & 0.1025 & 0.1697 & 0.3436 & 0.3310 \\ 0 & 0 & 0 & 0.1296 & 0.8704 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$p^{(4)} = [0.3907 \ 0.3294 \ 0.1827 \ 0.0828 \ 0.0144]$$

$$\boxed{P(X_4 = 4 \mid X_0 = 0) = 0.0144}$$

O elemento $(P^4)_{0,4}$ representa a probabilidade de o sistema sair do estado 0 e atingir o estado 4 em exatamente 4 ciclos.

1.5 d) Matriz Fundamental

Para cadeias absorventes, a matriz de transição é particionada como:

$$P = \begin{bmatrix} Q & R \\ 0 & I \end{bmatrix}$$

onde Q é a submatriz dos estados transientes e R a submatriz das transições ao estado absorvente:

$$Q = \begin{bmatrix} 0.7 & 0.3 & 0 & 0 \\ 0.2 & 0.5 & 0.3 & 0 \\ 0 & 0.1 & 0.5 & 0.4 \\ 0 & 0 & 0 & 0.6 \end{bmatrix} \quad R = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0.4 \end{bmatrix}$$

A matriz fundamental $N = (I - Q)^{-1}$, cujo elemento N_{ij} representa o número esperado de visitas ao estado j partindo do estado i antes da absorção:

$$N = (I - Q)^{-1} \approx \begin{bmatrix} 6.11 & 4.17 & 2.50 & 2.50 \\ 2.78 & 4.17 & 2.50 & 2.50 \\ 0.56 & 0.83 & 2.50 & 2.50 \\ 0 & 0 & 0 & 2.50 \end{bmatrix}$$

1.6 e) Número Médio de Ciclos até a Absorção

$$t = N \cdot \mathbf{1} = \begin{bmatrix} 15.28 \\ 11.94 \\ 6.39 \\ 2.50 \end{bmatrix}$$

- Partindo do estado 0: **15.28 ciclos**
- Partindo do estado 1: **11.94 ciclos**
- Partindo do estado 2: **6.39 ciclos**
- Partindo do estado 3: **2.50 ciclos**

1.7 f) Probabilidade de Absorção

$$B = N \cdot R = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

Como existe apenas um estado absorvente (estado 4), independentemente do estado inicial, o sensor será absorvido com probabilidade 1.

2 Exercício 2 — Simulação de Consultório Médico

2.1 Descrição do Sistema

Um consultório médico especializado em clínica geral atende pacientes ao longo do dia utilizando um sistema de agendamento combinado com encaixes de emergência. O consultório opera com disciplina de fila **FIFO** e possui os seguintes recursos:

- 1 recepcionista
- 1 sala de triagem
- 2 médicos

O fluxo do paciente segue a sequência: cadastro na recepção → triagem → aguarda atendimento médico → consulta → saída do sistema.

2.2 a) Identificação dos Componentes do Sistema

Entidades

- Pacientes (agendados e encaixes de emergência)

Recursos

- 1 recepcionista (responsável pelo cadastro)
- 1 sala de triagem
- 2 médicos (atendimento simultâneo possível)

Filas

- Fila da recepção: aguarda início do cadastro
- Fila da triagem: aguarda início da triagem
- Fila médica: aguarda início da consulta

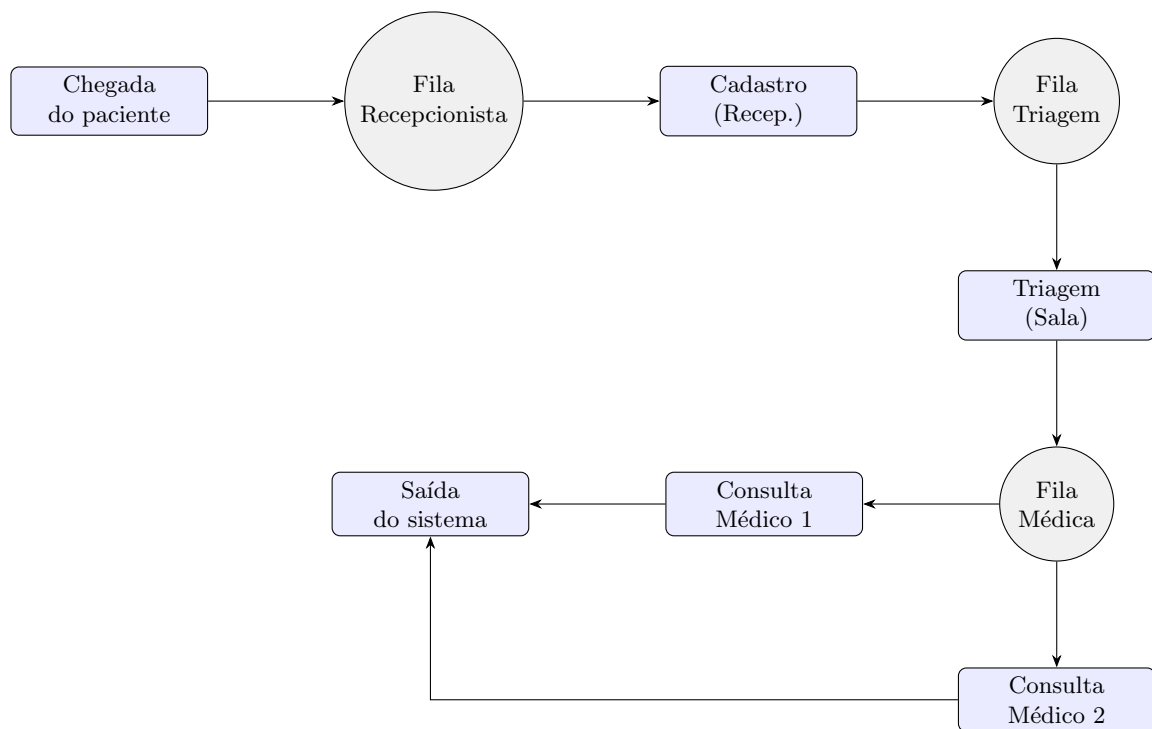
Eventos

- Chegada do paciente ao sistema
- Início e fim do cadastro
- Início e fim da triagem
- Início e fim da consulta médica
- Saída do sistema

Variáveis de Estado

- Número de pacientes em cada fila (recepção, triagem, médica)
- Estado da recepcionista: livre / ocupada
- Estado da sala de triagem: livre / ocupada
- Estado de cada médico: livre / ocupado (2 servidores independentes)

2.3 b) Diagrama ACD



2.4 c) Fases de Simulação (Abordagem de Três Fases)

Fase A — Avanço do Relógio

O relógio de simulação avança para o instante do próximo evento programado na lista de eventos futuros (FEL — *Future Event List*). Os eventos considerados são: chegadas, fins de cadastro, fins de triagem e fins de consulta.

Fase B — Eventos Incondicionais

Ocorrem independentemente do estado do sistema:

- **Chegada:** gera a próxima chegada e insere o paciente na fila da recepção.
- **Fim do cadastro:** libera a recepcionista e move o paciente para a fila de triagem.
- **Fim da triagem:** libera a sala de triagem e move o paciente para a fila médica.
- **Fim da consulta:** libera o médico e registra a saída do paciente.

Fase C — Eventos Condicionais

Dependem da disponibilidade de recursos:

- **Início do cadastro:** fila da recepção não vazia e recepcionista livre.
- **Início da triagem:** fila de triagem não vazia e sala de triagem livre.
- **Início da consulta:** fila médica não vazia e pelo menos um médico livre.

2.5 d) Dinâmica da Simulação

A dinâmica deste sistema baseia-se no método de **Simulação de Três Fases**, que garante que o estado do consultório seja atualizado de forma cronológica e logicamente consistente. A argumentação abaixo detalha como esses processos interagem:

Atualização do Relógio de Simulação (Avanço por Evento)

Diferente de uma simulação por intervalos de tempo fixos, este modelo utiliza o *Next-Event Time Advance*. O relógio "salta" diretamente para o instante do evento mais iminente na *Future Event List* (FEL). Esta abordagem é computacionalmente eficiente e garante que nenhum evento intermediário seja perdido, tratando o tempo como uma variável discreta definida pelas ocorrências de chegadas e fins de serviço.

Ativação de Servidores e Verificação de Recursos

A ativação dos servidores (recepcionista, sala de triagem e médicos) é regida por uma lógica de disponibilidade. Quando um evento de "Fim de Atividade" ocorre, o servidor não apenas se torna livre, mas dispara imediatamente uma verificação na fila correspondente. Se houver entidades aguardando, o início de uma nova atividade é agendado (Fase C), maximizando a taxa de utilização do recurso e reduzindo o tempo de espera ocioso.

Movimentação entre Filas e Sincronismo

A movimentação do paciente através do consultório é um processo de "empurrar e puxar" (*push-pull*):

- **Fluxo de Saída:** Ao concluir uma etapa (ex: cadastro), o paciente é "empurrado" para a próxima fila.
- **Fluxo de Entrada:** Se o recurso subsequente estiver livre, o paciente é "puxado" para o atendimento.

Este sincronismo é crítico no modelo, especialmente na transição da triagem para a consulta médica, onde a presença de dois médicos exige uma lógica de distribuição de carga (*load balancing*) para decidir qual servidor processará o próximo paciente da fila.

Verificação de Recursos Livres (Fase C)

A cada avanço do relógio, o sistema reavalia todas as condições de bloqueio. Um evento condicional (ex: iniciar consulta) só é disparado se a conjunção lógica (*Fila não vazia* $\wedge$ *Recurso disponível*) for verdadeira. No caso dos médicos, a regra de decisão verifica primeiro a disponibilidade do Médico 1 e, alternativamente, do Médico 2, garantindo o processamento paralelo que caracteriza este sistema multiserver.

2.6 e) Tabela de Simulação Manual

Dados fornecidos:

	P1	P2	P3	P4	P5
Entre chegadas (min)	5	7	3	10	6
Cadastro (min)	4	5	3	4	6
Triagem (min)	6	5	7	4	6
Consulta (min)	12	18	10	20	15

Table 1: Tempos de serviço por paciente

Paciente	Chegada	Início Cadastro	Fim Cadastro	Início Triagem	Fim Triagem	Início Consulta	Fim Consulta
P1	5	5	9	9	15	15	27
P2	12	12	17	17	22	22	40
P3	15	17	20	22	29	29	39
P4	25	25	29	29	33	39	59
P5	31	31	37	37	43	43	58

Table 2: Tabela de simulação manual — tempos em minutos

Nota: Os 2 médicos permitem atendimentos paralelos. P1 ocupa o médico 1 a partir de $t = 15$ e P2 ocupa o médico 2 a partir de $t = 22$. P3 inicia consulta no médico 1 em $t = 29$, assim que este fica livre.

2.7 f) Cálculo das Métricas

Tempo Médio na Fila da Recepção

$$\text{Espera na fila da Recepção} = [0 \quad 0 \quad 2 \quad 0 \quad 0]$$

$$W_{\text{fila_rec}} = \frac{(5 - 5) + (12 - 12) + (17 - 15) + (25 - 25) + (31 - 31)}{5} = \frac{0 + 0 + 2 + 0 + 0}{5} = 0,4 \text{ min}$$

Tempo Médio na Fila Médica

$$\text{Espera na fila Médica} = [0 \quad 0 \quad 0 \quad 6 \quad 0]$$

$$W_{\text{fila_med}} = \frac{(15 - 15) + (22 - 22) + (29 - 29) + (39 - 33) + (43 - 43)}{5} = \frac{0 + 0 + 0 + 6 + 0}{5} = 1,2 \text{ min}$$

Tempo Médio Total no Sistema

$$\text{Tempo no Sistema} = [22 \quad 28 \quad 24 \quad 34 \quad 27]$$

$$W_{\text{total}} = \frac{(27 - 5) + (40 - 12) + (39 - 15) + (59 - 25) + (58 - 31)}{5} = \frac{22 + 28 + 24 + 34 + 27}{5} = 27 \text{ min}$$

Taxa de Utilização dos Médicos

$$\rho_{\text{médicos}} = \frac{\text{tempo total de consultas}}{n_{\text{médicos}} \times T_{\text{simulação}}} = \frac{12 + 18 + 10 + 20 + 15}{2 \times 59} = \frac{75}{118} \approx 63,56\%$$

Número Médio de Pacientes no Sistema

Pelo Teorema de Little ($L = \lambda W$):

$$\lambda = \frac{5 \text{ pacientes}}{59 \text{ min}} \approx 0,0847 \text{ pac/min}$$

$$L = \lambda \times W_{\text{total}} = 0,0847 \times 27 \approx 2,29 \text{ pacientes}$$

2.8 Discussão dos Resultados

- **Eficiência da Recepção:** O tempo médio de espera de 0,4 min indica que o recurso de recepção é adequado para a carga de trabalho. O pequeno gargalo observado no Paciente 3 (2 min de espera) ocorre pela sobreposição pontual com o Paciente 2, mas o sistema se recupera rapidamente.
- **Dinâmica da Fila Médica:** A fila médica apresentou a maior espera relativa (1,2 min em média), com um pico de 6 minutos para o Paciente 4. Isso demonstra que, embora existam dois médicos, a alta variabilidade nos tempos de consulta (especialmente a consulta de 18 min do P2) pode causar retenções temporárias quando ambos os profissionais estão ocupados simultaneamente.
- **Ocupação dos Recursos:** A taxa de utilização de 63,56% revela um sistema equilibrado. Uma utilização nesta faixa permite que os médicos absorvam a variabilidade natural das consultas sem que as filas cresçam de forma instável, garantindo um tempo total no sistema (27 min) composto majoritariamente por tempo de atendimento efetivo.
- **Dimensionamento Físico:** O número médio de 2,29 pacientes no sistema sugere que o consultório mantém, em média, entre 2 e 3 pessoas circulando. Este dado é fundamental para o planejamento da infraestrutura, indicando que o espaço físico e a quantidade de assentos na sala de espera estão bem dimensionados para este fluxo de pacientes.

A Implementação em Python — Exercício 1

A seguir apresenta-se o código Python desenvolvido para o item g), organizado em cinco blocos correspondentes às células do Jupyter Notebook entregue.

Célula 1 — Imports e Matriz de Transição

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.patches as mpatches
4 from matplotlib.patches import FancyArrowPatch
5
6 # Matriz de transicao P onde P[i,j] representa a probabilidade
7 # de transicao do estado i para o estado j em um ciclo
8 P = np.array([
9     [0.7, 0.3, 0.0, 0.0, 0.0], # Do estado 0: permanece(0.7), vai p/1(0.3)
10    [0.2, 0.5, 0.3, 0.0, 0.0], # Do estado 1: volta p/0(0.2), permanece(0.5), vai p
11    /2(0.3)
12    [0.0, 0.1, 0.5, 0.4, 0.0], # Do estado 2: volta p/1(0.1), permanece(0.5), vai p
13    /3(0.4)
14    [0.0, 0.0, 0.0, 0.6, 0.4], # Do estado 3: permanece(0.6), vai p/4(0.4)
15    [0.0, 0.0, 0.0, 0.0, 1.0], # Do estado 4: absorvente, permanece sempre(1.0)
16 ])
17
18 print("\b) Matriz de transicao P:")
19 print(P)
```

Célula 2 — Probabilidades: itens c), d), e), f)

```
1 # c) Probabilidade de atingir o estado 4 exatamente no 4o ciclo
2 # Elevamos a matriz P a potencia 4 para obter as probabilidades
3 # de transicao em exatamente 4 ciclos. O elemento P^4[0,4]
4 # fornece a probabilidade de partir do estado 0 e chegar ao estado 4
5 # exatamente no 4o ciclo.
6 P4 = np.linalg.matrix_power(P, 4)
7 print("\c) Probabilidade de atingir o estado 4 exatamente no 4o ciclo")
8 print(f"    P(X4=4 | X0=0) = P^4[0,4] = {round(P4[0, 4], 6)}")
9
10 # d) Matriz fundamental N = (I - Q)^-1
11 # Para cadeias de Markov absorventes, separamos a matriz P em:
12 #   Q: submatriz dos estados transientes entre si (estados 0,1,2,3)
13 #   R: submatriz das transicoes dos estados transientes ao absorvente (estado 4)
14 # A matriz fundamental N = (I - Q)^-1 indica o numero esperado de vezes
15 # que a cadeia visita cada estado transiente j, dado que inicia no estado i.
16 Q = P[:4, :4] # Submatriz 4x4 dos estados transientes
17 R = P[:4, 4:] # Submatriz 4x1 das transicoes para o estado absorvente
18
19 N = np.linalg.inv(np.eye(4) - Q) # Matriz fundamental
20
21 print("\nd) Matriz fundamental N = (I - Q)^-1:")
22 print(np.round(N, 4))
23
24 # e) Numero medio de ciclos ate a absorcao
25 # O vetor t = N*1 fornece o numero medio de ciclos que o sensor
26 # permanece em funcionamento antes de ser absorvido pelo estado 4.
27 t = N @ np.ones(4)
28 print("\ne) Numero medio de ciclos ate a absorcao por estado inicial:")
29 for i, ti in enumerate(t):
30     print(f"    Partindo do estado {i}: {ti:.2f} ciclos em media")
31
32 # f) Probabilidade de absorcao no estado 4
33 # O vetor B = N*R fornece a probabilidade de o sensor ser absorvido
34 # pelo estado 4, para cada estado inicial.
35 B = N @ R
36 print("\nf) Probabilidade de absorcao no estado 4 por estado inicial:")
37 for i, bi in enumerate(B):
38     print(f"    Partindo do estado {i}: {bi[0]:.6f}")
```


Célula 3 — Simulação da Cadeia

```
1 # Fixamos a semente para garantir reprodutibilidade dos resultados
2 np.random.seed(42)
3
4 # Parametros da simulacao
5 estado_atual = 0 # Sensor inicia totalmente operacional (estado 0)
6 num_ciclos = 30 # Numero de ciclos de operacao simulados
7 historico = [estado_atual]
8
9 # A cada ciclo, o proximo estado e sorteado de acordo com
10 # a linha da matriz P correspondente ao estado atual
11 for _ in range(num_ciclos):
12     proximo = np.random.choice(5, p=P[estado_atual])
13     historico.append(proximo)
14     estado_atual = proximo
15
16 print("Historico de estados simulados:")
17 print(historico)
18 print(f"Estado final apos {num_ciclos} ciclos: {historico[-1]}")
19
20 cores_estados = ['#2ecc71', '#3498db', '#f39c12', '#e74c3c', '#8e44ad']
21 rotulos = ['0-Operacional', '1-Degradacao', '2-Falha mod.', '3-Critico', '4-Queimado']
22
23 # Grafico do trajeto simulado:
24 # A linha em degraus mostra a evolucao do estado ao longo dos ciclos.
25 # Os pontos coloridos identificam o estado em cada instante.
26 plt.figure(figsize=(14, 4))
27 plt.step(range(len(historico)), historico, where='post',
28         color='#378ADD', linewidth=2, zorder=2)
29 plt.scatter(range(len(historico)), historico,
30            c=[cores_estados[s] for s in historico],
31            zorder=5, s=60, edgecolors='white', linewidths=0.8)
32
33 plt.xlabel("Ciclo", fontsize=12)
34 plt.ylabel("Estado", fontsize=12)
35 plt.title("g) Simulacao da Cadeia de Markov - trajeto do sensor", fontsize=13)
36 plt.yticks([0,1,2,3,4], rotulos)
37 plt.xticks(range(0, num_ciclos + 2, 2))
38 plt.grid(axis='x', alpha=0.3)
39 plt.tight_layout()
40 plt.show()
```

Célula 4 — Evolução Temporal das Probabilidades

```
1 # O vetor inicial representa o sensor partindo com certeza do estado 0
2 prob_inicial = np.array([1.0, 0.0, 0.0, 0.0, 0.0])
3 n_passos = 20
4 probabilidades = [prob_inicial]
5
6 # A cada passo, multiplicamos o vetor de probabilidades pela matriz P:
7 #  $pi(n+1) = pi(n) * P$ 
8 # Isso fornece a distribuicao de probabilidade dos estados apos n ciclos.
9 for _ in range(n_passos):
10     probabilidades.append(probabilidades[-1] @ P)
11
12 probabilidades = np.array(probabilidades)
13
14 cores_plot = ['#2ecc71', '#3498db', '#f39c12', '#e74c3c', '#8e44ad']
15 estilos = ['-', '--', '-.', ':', '-']
16 rotulos_est = [
17     'Estado 0 - Operacional',
18     'Estado 1 - Degradacao leve',
19     'Estado 2 - Falha moderada',
20     'Estado 3 - Critico',
21     'Estado 4 - Queimado (absorvente)',
22 ]
23
24 # Grafico da evolucao temporal:
25 # Cada curva mostra como a probabilidade de estar em cada estado
26 # evolui ao longo dos ciclos. A probabilidade do estado 4
```

```

27 # converge para 1, pois e o unico estado absorvente.
28 plt.figure(figsize=(12, 5))
29 for estado in range(5):
30     plt.plot(probabilidades[:, estado],
31             label=rotulos_est[estado],
32             color=cores_plot[estado],
33             linestyle=estilos[estado],
34             linewidth=2,
35             marker='o', markersize=4)
36
37 plt.xlabel("Ciclos", fontsize=12)
38 plt.ylabel("Probabilidade", fontsize=12)
39 plt.title("g) Evolucao temporal das probabilidades dos estados", fontsize=13)
40 plt.legend(loc='center right', fontsize=9)
41 plt.grid(alpha=0.3)
42 plt.tight_layout()
43 plt.show()

```

Célula 5 — Grafo de Transições

```

1 # Cores que diferenciam visualmente o tipo de transicao
2 COR_FORWARD = "#378ADD" # Azul - transicoes para estado pior (degradacao)
3 COR_BACKWARD = "#D85A30" # Coral - transicoes para estado melhor (recuperacao)
4 COR_LOOP = "#BA7517" # Ambar - permanencia no mesmo estado (self-loop)
5 COR_ABS = "#EF9F27" # Laranja - estado absorvente (queimado)
6
7 pos = {0: (1,0), 1: (3,0), 2: (5,0), 3: (7,0), 4: (9,0)}
8 R_no = 0.28
9
10 fig, ax = plt.subplots(figsize=(18, 6))
11 ax.set_xlim(0, 10)
12 ax.set_ylim(-1.0, 1.4)
13 ax.set_aspect('equal')
14 ax.axis("off")
15
16 def draw_arrow(ax, x1, y1, x2, y2, rad, color):
17     """Desenha uma seta curva entre dois pontos usando FancyArrowPatch.
18     O parametro rad controla o grau e sentido da curvatura."""
19     ax.add_patch(FancyArrowPatch(
20         posA=(x1, y1), posB=(x2, y2),
21         connectionstyle=f"arc3,rad={rad}",
22         arrowstyle="-|>",
23         mutation_scale=18,
24         linewidth=2,
25         color=color,
26     ))
27
28 def ponto_no_circulo(cx, cy, angulo_graus):
29     """Calcula as coordenadas de um ponto na borda do no circular,
30     para que as setas partam/cheguem na borda e nao no centro."""
31     a = np.radians(angulo_graus)
32     return cx + R_no * np.cos(a), cy + R_no * np.sin(a)
33
34 # Transicoes para frente (degradacao): curva acima dos nos
35 for i in range(5):
36     for j in range(5):
37         if P[i, j] > 0 and i < j:
38             x1, _ = pos[i]; x2, _ = pos[j]
39             sx, sy = ponto_no_circulo(x1, 0, 80)
40             tx, ty = ponto_no_circulo(x2, 0, 100)
41             draw_arrow(ax, sx, sy, tx, ty, rad=-0.25, color=COR_FORWARD)
42             dist = x2 - x1
43             peak_y = 0.28 + 0.13 * dist
44             ax.text((x1+x2)/2, peak_y, str(round(P[i,j],2)),
45                 ha='center', va='bottom', fontsize=9, color=COR_FORWARD)
46
47 # Transicoes para tras (recuperacao): curva abaixo dos nos
48 for i in range(5):
49     for j in range(5):
50         if P[i, j] > 0 and i > j:
51             x1, _ = pos[i]; x2, _ = pos[j]

```

```

52         sx, sy = ponto_no_circulo(x1, 0, -100)
53         tx, ty = ponto_no_circulo(x2, 0, -80)
54         draw_arrow(ax, sx, sy, tx, ty, rad=-0.25, color=COR_BACKWARD)
55         dist = x1 - x2
56         peak_y = -0.28 - 0.13 * dist
57         ax.text((x1+x2)/2, peak_y, str(round(P[i,j],2)),
58                ha='center', va='top', fontsize=9, color=COR_BACKWARD)
59
60     # Self-loops (permanencia): arco compacto acima do no
61     for i in range(5):
62         if P[i, i] > 0:
63             x, y = pos[i]
64             sx, sy = ponto_no_circulo(x, y, 120)
65             tx, ty = ponto_no_circulo(x, y, 60)
66             ax.add_patch(FancyArrowPatch(
67                 posA=(sx, sy), posB=(tx, ty),
68                 connectionstyle="arc3,rad=-0.7",
69                 arrowstyle="-|>",
70                 mutation_scale=18,
71                 linewidth=2,
72                 color=COR_LOOP,
73             ))
74             ax.text(x, y + 0.60, str(round(P[i,i],2)),
75                    ha='center', va='bottom', fontsize=9, color=COR_LOOP)
76
77     # Nos desenhados por ultimo para ficarem sobre as setas
78     for i, (x, y) in pos.items():
79         cor = COR_ABS if i == 4 else COR_FORWARD
80         ax.add_patch(plt.Circle((x, y), R_no, color=cor, zorder=5))
81         ax.text(x, y, str(i), ha='center', va='center',
82                color='white', fontweight='bold', fontsize=13, zorder=6)
83
84     ax.legend(handles=[
85         mpatches.Patch(color=COR_FORWARD, label="Para frente (degradacao)",
86         mpatches.Patch(color=COR_BACKWARD, label="Para tras (recuperacao)",
87         mpatches.Patch(color=COR_LOOP, label="Self-loop (permanencia)",
88         mpatches.Patch(color=COR_ABS, label="Estado absorvente (queimado)"),
89     ], loc="upper right", fontsize=9)
90
91     plt.title("g) Grafo de transicoes da Cadeia de Markov", fontsize=14)
92     plt.tight_layout()
93     plt.show()

```